S&P 500 Lakehouse: Architecture and FinOps Report

Haider Ali | 11 November 2026 | github.com/thehaiderali/dav-medallion-pipeline at commit 5d3c276

Runtime: Databricks Free Edition, AWS us-east-2 (Ohio). Serverless jobs (notebook task) and Serverless Starter SQL warehouse (2X-Small, auto-stop 10 min).

1. Architecture

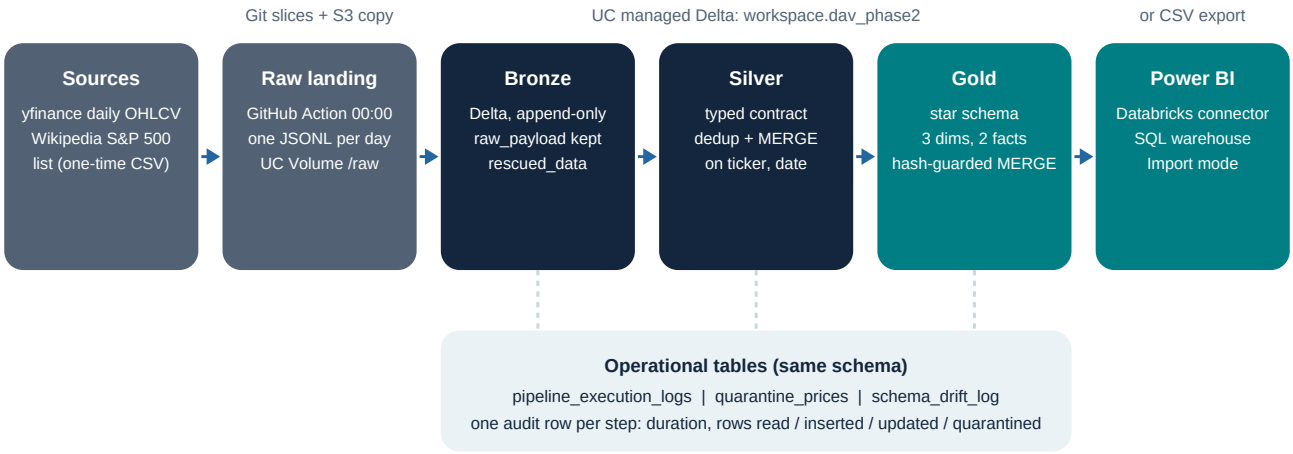

Figure 1. Data flow. Bronze, Silver and Gold run as one serverless notebook job (00_end_to_end). Each Silver batch triggers the Gold refresh.

A GitHub Action extracts one trading day at midnight and commits it as a date-named JSONL slice; the 4-year history was extracted once into a single file. Raw files are copied into a Unity Catalog Volume. Bronze stores every arrival with its original JSON. Silver casts to an explicit contract, sends invalid rows to quarantine, keeps the latest record per (ticker, trade_date) and MERGEs it. Gold publishes a star schema that Power BI imports through a SQL warehouse.

2. What was run

Run	Input	Result	Wall time
Full load	full_load.jsonl, 498,238 rows, 2022-10-03 to 2026-09-30	SUCCESS. 498,238 rows in Silver and Gold, 0 quarantined	170 s
Incremental backfill	17 daily slices, 2026-10-01 to 2026-10-23	All 17 days reached Gold. Job marked FAILED by 6 weekend dates (see 3.6)	831 s

Final table	Rows	Check
silver_prices / gold_fact_daily_price	506,784	498,238 + 12 x 503 + 5 x 502; 0 duplicate (ticker, date) keys
gold_fact_sector_daily	11,209	11,022 + 17 days x 11 sectors
gold_dim_date / company / sector	1,019 / 503 / 11	trading dates 2022-10-03 to 2026-10-23
quarantine_prices / schema_drift_log	0 / 0	no invalid values or new source fields in this data

3. Decision log

3.1 Partitioning

Raw files are partitioned by date; Delta tables are not. Incremental input lives at incremental/YYYY-MM-DD/data.jsonl, so a rerun or backfill addresses exactly one day and the batch id (incremental_YYYYMMDD) is derived from the path. No Delta table has partition columns:

Table	Files	Size	Partition columns
bronze_prices	5	56.6 MB	none
silver_prices	1	26.3 MB	none
gold_fact_daily_price	25	32.6 MB	none
gold_fact_sector_daily	1	0.2 MB	none

Partitioning Silver by trade_date would create 1,019 directories of roughly 25 KB each. That is the small-file pattern Delta guidance warns against: more file listing and metadata work for no I/O saving. Date filters are still selective without partitions because files are written in date order and Delta keeps min/max statistics per file. Measured: a one-day filter on gold_fact_daily_price skipped 24 of 26 files (section 4). Partitioning or liquid clustering on date_key becomes worthwhile when a table reaches hundreds of GB.

3.2 Schema drift

Bronze reads each JSON line as text and parses it twice: once against the price contract and once as a string map. Keys outside the contract are kept three ways: inside raw_payload, in a rescued_data JSON column, and as new string columns added with mergeSchema. Each new key writes a NEW_COLUMN row to schema_drift_log. Silver selects only contract columns with try_cast, so a new field never breaks Silver or Gold, and a changed type sends the row to quarantine with a reject reason instead of failing the batch.

No drift occurred in the production data: schema_drift_log has 0 rows after 515,330 Bronze rows. The added-column path is exercised by notebook 03_demo_idempotency_and_audit, which was not executed for this report. One source change did occur and is not schema drift: KVUE has no prices after 2026-10-16, so daily slices dropped from 503 to 502 rows. The pipeline accepted this; the company dimension keeps KVUE and its history.

3.3 PII

The data contains no personal data: public daily prices and a public company directory. Audit tables store run ids, batch ids, Volume paths and counts, with no user names or e-mail addresses. The decision was therefore not to add masking. If a person-level field appeared it would surface first as a NEW_COLUMN drift row; the response would be to drop or hash it in the Bronze-to-Silver projection and apply a Unity Catalog column mask to the Bronze copy.

3.4 Idempotent loads

Bronze is append-only so every arrival stays auditable. Silver and Gold MERGE on natural keys and update only when a row hash changes. This was tested by accident: a cancelled automatic retry re-appended 8,546 rows to Bronze, and Silver and Gold still hold 506,784 rows with zero duplicate keys.

3.5 Gold refresh after every Silver batch

Refreshing Gold per batch keeps Gold consistent with Silver at every commit and costs little for one daily batch. In a backfill it repeats five Gold steps per day: about 23 of the 33.2 s spent per trading day. Refreshing Gold once at the end of a window gives the same result, because the Gold MERGEs are hash-guarded.

3.6 Known defect: skip_missing on serverless

Weekend dates have no file. raw_to_bronze catches a missing-path error to skip them, but serverless (Spark Connect) analyses the read lazily, so the error is raised after the try block. The six weekend dates were logged as FAILURE and the job failed after all trading days had loaded. Fix: check the path with dbutils.fs or force analysis inside the try block.

4. Spark execution of the Gold sector fact

The most expensive Gold step is SILVER_TO_GOLD_SECTOR_FACT. It joins every daily price row to the company dimension to get a sector, groups by (date_key, sector_key), and overwrites gold_fact_sector_daily. The same query was run on the 2X-Small serverless SQL warehouse to capture its plan and metrics.

Screenshot not included. Serverless compute has no classic Spark UI; the equivalent is the Query Profile in SQL query history. Save it as docs/screenshots/spark_dag_sector_fact.png (statement 01f1dd3f-7bb3-11e3-af9d-9acf06fc9e44) and rebuild this PDF to embed it.

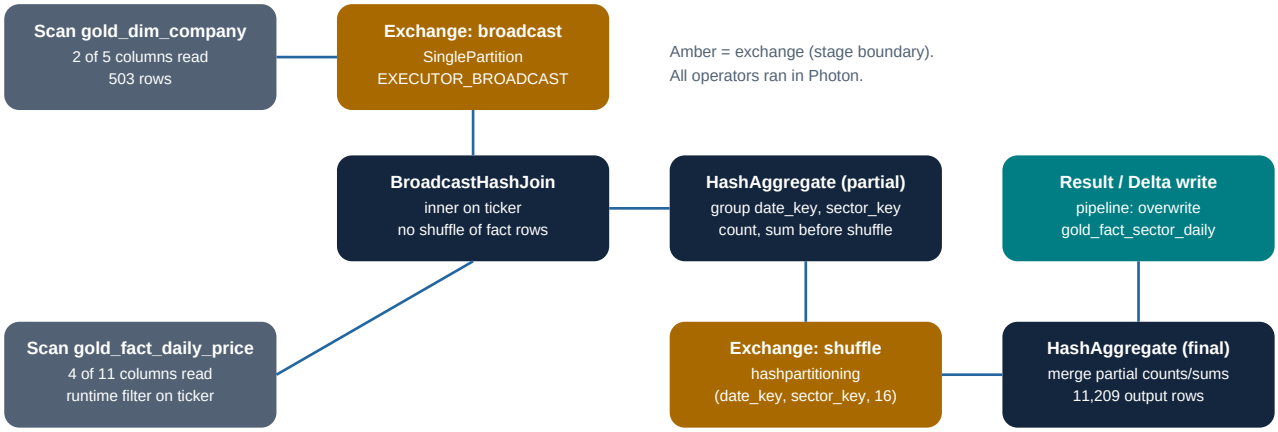

Figure 3. Physical plan of the sector aggregation, transcribed from EXPLAIN FORMATTED on the SQL warehouse.

How Spark executed it

Broadcast instead of shuffle join. The company table is 503 rows, so the optimizer collects it to one partition and broadcasts it to every executor (EXECUTOR_BROADCAST). The 507k fact rows are joined in place and never shuffled for the join.

Column pruning and runtime filtering. The fact scan reads 4 of its 11 columns (date_key, ticker, volume, dollar_volume_proxy), so only 9.1 MB of the 32.6 MB table was read. The scan also carries an OptionalDataFilter, hashedrelationcontains(ticker), built from the broadcast side; it drops rows with no company before the join.

One shuffle, after a partial aggregate. Each task pre-aggregates its rows by (date_key, sector_key). Only those partial counts and sums cross the network, hash-partitioned into 16 shuffle partitions, and a final aggregate merges them. This exchange is the single stage boundary in the fact path. Adaptive Query Execution is on, and Photon ran every operator.

Predicate pushdown. Adding WHERE date_key = 20261023 pushes the filter into the Delta scan. File statistics then exclude files whose date_key range cannot match:

Metric	Full rebuild	One-day filter
Files read / skipped	26 / 0	2 / 24
Rows read	507,287	1,004
Bytes read	9.06 MB	62.8 KB
Tasks / total task time	10 / 3.3 s	3 / 0.04 s
Wall time	1.80 s	0.88 s
Spill to disk	0	0

In the pipeline this step averaged 3.6 s per run, and it rebuilds every date each time. The step also calls count() on the joined frame and on the result before writing, so the join and aggregation execute three times per refresh instead of once.

5. Cost and FinOps

5.1 What the runs consumed

The workspace is Databricks Free Edition, so the actual spend was USD 0. The project did not use Azure for Students credit; the workspace runs on Databricks-managed AWS (us-east-2). Usage still appears in system.billing.usage, but that table was last ingested on 2026-10-15 and does not yet contain the 10 November jobs. Up to that date it recorded 1.70 DBU of serverless interactive compute (development sessions) and 0.31 DSU of storage. The cost figures below therefore combine measured run times with list prices from system.billing.list_prices for this region:

SKU (us-east-2)	List price
Jobs serverless compute	USD 0.35 / DBU
All-purpose serverless compute (interactive notebooks)	USD 0.75 / DBU
Serverless SQL (Power BI queries)	USD 0.70 / DBU

5.2 A typical daily incremental run

A single-day run was not timed on its own. It is derived from the backfill, which processed 17 trading days in one 808 s invocation: **47 s per trading day** (of which 33.2 s is inside the seven audited steps and 14 s is between steps, mainly the audit-log append after each step), 6.6 s of metadata steps, and 23 s of job start-up and notebook overhead. Total: **about 76 s (1.3 min) of serverless compute per daily run**. The full load, for comparison, took 170 s for 498,238 rows: run time is driven by the number of Delta operations, at 4 to 7 s each, rather than by row count.

5.3 Hourly for a year

Databricks does not publish a fixed DBU-per-hour rate for serverless jobs, so the table shows three consumption scenarios at the jobs list price:

Schedule	Runs / year	Compute hours	2 DBU/h	4 DBU/h	8 DBU/h
Daily (current)	365	7.7	USD 5.42	USD 10.85	USD 21.70
Hourly	8,760	186.0	USD 130	USD 260	USD 521

The pipeline is not the main cost; the BI refresh is. A 2X-Small serverless SQL warehouse uses 4 DBU/h (USD 2.80/h) and stays up for its 10-minute auto-stop after each refresh. Refreshing Power BI hourly keeps it running about 1,606 h a year: **USD 4,497**, against USD 187 for a daily refresh. Storage for the 116 MB of Delta tables is under USD 1 a month.

Hourly enterprise estimate: about USD 4,757 a year at the 4 DBU/h scenario, before retries and egress. A daily schedule costs about USD 198.

5.4 Optimizations, in order of effect

Change	Basis
Do not run hourly. Trigger the job when the daily slice lands (file-arrival trigger on the Volume) and refresh Power BI once after it succeeds.	yfinance provides end-of-day bars, so 23 of 24 hourly runs would find no new data. Removes about 96% of both compute lines.
Refresh Gold once per invocation, not once per batch.	Five Gold steps take about 23 s of the 33.2 s measured per trading day; a 17-day backfill would drop from 17 to 1 Gold refresh.
Aggregate only the changed dates into gold_fact_sector_daily and MERGE them.	Measured: a one-day filter reads 1,004 rows from 2 files instead of 507,287 from 26. Also drop the two count() actions.
Write audit rows once per invocation instead of after every step.	About 14 s per trading day is spent outside the audited steps.
Compact gold_fact_daily_price weekly (OPTIMIZE).	Daily MERGEs left 25 files for 32.6 MB.

Sources: system.billing.list_prices and system.billing.usage in this workspace; Databricks serverless SQL warehouse sizing documentation (2X-Small = 4 DBU/h). Measured values are in docs/evidence/phase3_run_evidence.json.